

Logistic Regression

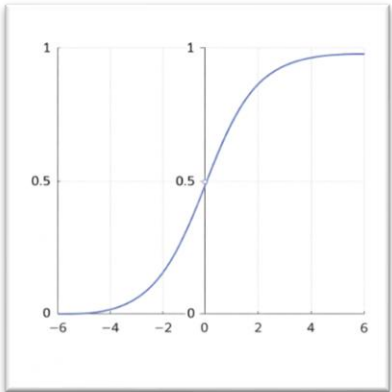
Overview

- a **machine learning** task where we **predict** a real-valued **output** response given an **input vector**
- **linear** and **non-linear** regression **models**
- **analytic** (closed-form) **solution** to learn an **optimal model** from **training data**
- **gradient descent** and **stochastic gradient** descent algorithms
- model **complexity** and **overfitting**

Readings

- <https://www.microsoft.com/en-us/research/wp-content/uploads/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf#page=157>

Async	
Logistic Regression	
Definition	• supervised learning technique used to train models for classification where y is categorical • generally, links to a binary link; is $x[a, b, c, \dots]$ likely to produce y • $x \in \mathbb{R}^d \Rightarrow y \in \{1, \dots, C\}$
Fitting the Model	• maximize the likelihood of y given x • $P_{\theta}(y = 1 x) = \sigma(\varphi(x)^T \theta) = \frac{1}{(1 + \exp(-\varphi(x)^T \theta))}$ • to maximize the likelihood by minimizing the negative log likelihood (NLL) • $\operatorname{argmax}_{\theta} \prod_{i=1}^n P_{\theta}(y_i x_i) =$ $\operatorname{argmin}_{\theta} - \frac{1}{n} \sum_{i=1}^n \log P_{\theta}(y_i x_i)$ • putting the likelihood of y given x into terms of the regression model ○ for $y = 1$, we need the probability of $y = 1$ given x given by the sigmoid of the linear function: $\sigma(\theta^T x_i)$ ○ for $y = 0$, we need the probability of $y = 0$ given x is given by $1 - \sigma(\theta^T x_i)$ ○ which implies an 'if' statement in our algorithm ○ so, $\log p_{\theta}(y_i x_i) = \begin{cases} \log \sigma(\theta^T x_i) & \text{if } y_i = 1 \\ \log(1 - \sigma(\theta^T x_i)) & \text{if } y_i = 0 \end{cases}$ ○ utilizing indicator values as coefficients, we can effectively insert a Boolean 'if' in the equation ○ so, $\log p_{\theta}(y_i x_i) =$ $\{y_i = 1\} \log \sigma(\theta^T x_i) +$

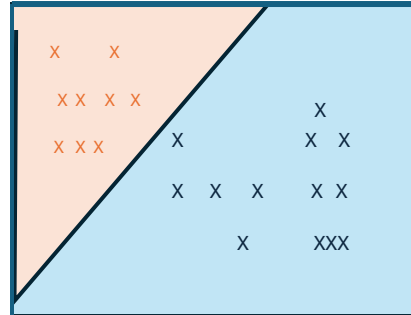
	$\mathbb{1}\{y_i = 0\} \log(1 - \sigma(\theta^T x_i))$ given $\begin{cases} \mathbb{1} = 1 & \text{if True} \\ \mathbb{1} = 0 & \text{if False} \end{cases}$ - and finally, “after some algebra”, we conclude that $\arg \min_{\theta} - \frac{1}{n} \sum_{i=1}^n (y_i \phi(x_i)^T \theta + \log(\sigma(-\phi(x_i)^T \theta)))$ is equivalent to minimizing a KL divergence known as the cross-entropy loss - this justifies the binary adjustment to the classification problem
Fitting the Modes	
Logistic Regression Model	$P_{\theta}(y = 1 x) = \sigma(\phi(x)^T \theta) = \frac{1}{1 + \exp(-\phi(x)^T \theta)}$ 
Maximize the Likelihood	- as before, we maximize the likelihood of our data by minimizing the negative log likelihood $\arg \max_{\theta} \prod_{i=1}^n \widehat{P}_{\theta}(y_i x_i) =$ $\arg \min_{\theta} - \frac{1}{n} \sum_{i=1}^n \log \widehat{P}_{\theta}(y_i x_i)$
(Average) Negative Log Likelihood	$\log p_{\theta}(y_i x_i) = \begin{cases} \log \sigma(\theta^T x_i) & y_i=1 \\ \log(1 - \sigma(\theta^T x_i)) & y_i=0 \end{cases}$ - to optimize according to “if statements” as above: $\log p_{\theta}(y_i x_i) = IF\{y_i = 1\} \log \sigma(\theta^T x_i) + IF\{y_i = 0\} \log(1 - \sigma(\theta^T x_i))$

	$= y_i \log \sigma(\theta^T x_i) + (1 - y_i) \log(1 - \sigma(\theta^T x_i))$
Minimizing the Loss	
Logistic Loss Function	• average KL divergence (simplified) $\arg \min_{\theta} -\frac{1}{n} \sum_{i=1}^n (y_i \log \sigma(\phi(x_i)^T \theta) + \log \sigma(-\phi(x_i)^T \theta))$ • take derivative $\nabla_{\theta} L(\theta) = -\frac{1}{n} \sum_{i=1}^n (y_i \nabla_{\theta} \sigma(\phi(x_i)^T \theta) + \nabla_{\theta} \log \sigma(-\phi(x_i)^T \theta))$ • no general analytic solution • solved using numerical methods
Useful Identity	• $\frac{\partial}{\partial t} \sigma(t) = \sigma(t)(1 - \sigma(t))$
Gradient Descent	
The G. D. Algorithm	• $\theta^0 \leftarrow$ initial vector (random, zeros, ...) • for τ from 0 to convergence $\theta^{\tau+1} \leftarrow \theta^{\tau} - \rho(\tau) (\nabla_{\theta} L(\theta)) _{\theta=\theta^{\tau}}$ • $\rho(\tau)$ is the step size (learning rate) ◦ typically, $\frac{1}{\tau}$ • converges when gradient is ≈ 0
Stochastic Gradient Descent	• parameters are updated using one or small batches • for many learning problems, the gradient is a sum $\nabla_{\theta} L(\theta) = \frac{1}{n} \sum_{i=1}^n (\sigma(\phi(x_i)^T \theta) - y_i) \phi(x_i)$ ◦ for large n this can be costly • approximate the gradient by looking at a few random points • $\nabla_{\theta} L(\theta) \approx \frac{1}{ B } \sum_{i \in B} (\sigma(\phi(x_i)^T \theta) - y_i) \phi(x_i)$  batch size  random sample of records

Linearly Separable Data

Definition

- a classification dataset is said to be linearly separable if there exists a hyperplane that separates the two classes



Solution

- if data is linearly separable, logistic regression requires regularization
- this is achieved by “tacking on” a penalty for the magnitude of θ

$$+ \lambda \sum_{j=1}^d \theta_j^2$$

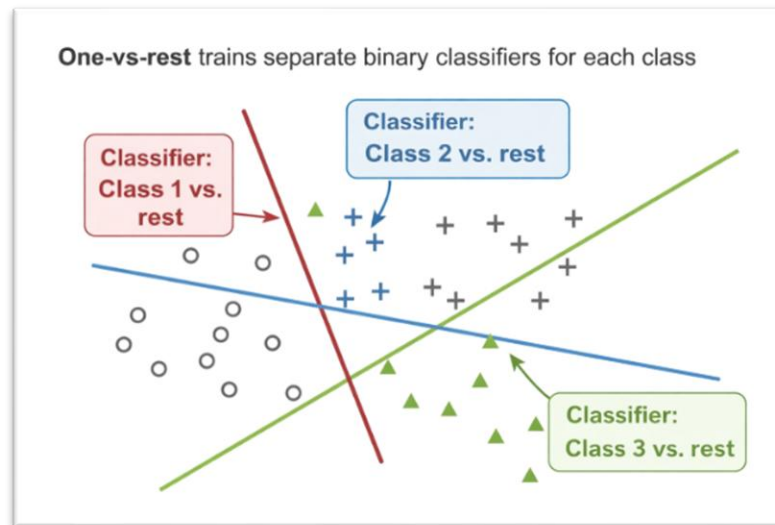
- prevents weights from diverging on linearly separable data
- prevents overfitting

Scikit-Learn

```
from sklearn.linear_model import LogisticRegression
# sklearn adds regularization
# C = 1 / by default lambda the inverse regularization
parameter
model = LogisticRegression(C = 100.0)
# Train the model
model.fit(df[['feat1', 'feat2']], df['label'])
# Make predictions
test_df['label'] = model.predict(test_df[['feat1',
'feat2']])
test_df['P(Y|X)'] = model.predict_proba(test_df[['feat1',
'feat2']])
```

**Binary
Classification of
Categorical Data**

- for Y in $\{1, \dots, C\}$ you **train one** binary **classifier** per class by **grouping one classifier against all others** for each classifier



ChatGPT 5.2

**Softmax
multiclass
classification**

- $P(y = j | x) = \frac{\exp(x^T \theta^j)}{\sum_{k=1}^K \exp(x^T \theta_k)}$
- separate $\theta_j \in \mathbb{R}^d$ for each **class**
- **trained** using **GD** methods
- use **K-1 parameters**
 - $P(y = k | x) = 1 - \sum_{j \neq k} P(y = j | x)$
 - often use **K parameters + regularization** to address “redundancy”